

GPUDirect TCP: NIC-to-GPU DMA for a Fully GPU-Resident TCP/IP Stack

Ashish Ahuja	Aujasvit Datta	Darshan Dinesh Sethia	Divyansh Garg	Ritvik Goyal
220235	220254	220326	220377	220898
aahuja22@iitk.ac.in	aujasvitd22@iitk.ac.in	darshands22@iitk.ac.in	divyanshg22@iitk.ac.in	ritvikg22@iitk.ac.in

I. INTRODUCTION

This document describes GPUDirect TCP¹ (abbr. `gdrtcp`), a framework consisting of a fully GPU-resident TCP stack enabled by P2P NIC-to-GPU packet transfers. From the user's perspective, GPUDirect TCP consists of two parts:

- 1) A modified NIC driver in the form of a kernel module, which enables NIC to GPU DMA packet transfers.
- 2) A CUDA library which provides socket-like primitives allowing the user to create TCP connections and send/receive data through device-side code.

The CPU is on the critical path when running a GPU-centric workload on a remote GPU as it is required to transfer data from the NIC to the GPU. This entails performing unnecessary copies through system memory leading to an increase in memory bandwidth pressure and higher latency. Additionally, TCP/IP processing leads to a variety of adverse effects on the host such as cache pollution and high cpu utilization.

GPUDirect TCP aims to solve the above problems by enabling a DMA path to transfer packets from the NIC to the GPU as well as by implementing a complete TCP/IP stack on the GPU in device code. In essence, the GPU will communicate directly with its remote peer while keeping CPU involvement to a minimum.

The broad targets behind GPUDirect TCP are as follows:

- 1) The GPU workload must be able to connect to TCP servers to receive data at a high throughput with acceptable latency.
- 2) CPU involvement at all steps should be kept to a minimum.
- 3) The framework should be transparent and not impact the functionality or throughput of other applications running on the host.

Our hardware setup consists of two gigabit NICs: an Intel I219-LM running the `e1000e` driver, and an Intel I210 running the `igb` driver. Modifications are made to the `e1000e` driver, although the modifications should be easily extendable to other NIC drivers with similar structures. The GPU we have used is an NVIDIA Quadro P620; this is a fairly old GPU, for which we have no option but to use NVIDIA's proprietary device driver. All testing is done on the 6.1.4 version of the linux kernel.

The design is explained in four parts:

- 1) NIC to GPU DMA
- 2) Packet Redirection from GPU to CPU
- 3) Transmitting packets originating from the GPU
- 4) The GPU TCP/IP stack

II. NIC TO GPU DMA

Naturally, the first problem to tackle is DMA'ing packets from the NIC to the GPU. The DMA flow roughly looks as follows:

- 1) Pin some memory in GPU VRAM and expose it via PCI BARs.
- 2) Get the DMA address from the NIC's perspective (which could be different from the physical address due to IOMMUs).
- 3) Program the NIC's DMA engine with the DMA address.
- 4) Send notifications to the GPU after a packet has been DMA'd.

A. Setup

Driver support is essential to pin the GPU virtual memory into which the packets will be DMA'd, and to also expose it via PCI BARs. Fortunately, NVIDIA introduced GPUDirect RDMA in Kepler-class GPUs that "enables a direct path for data exchange between the GPU and a third-party peer device using standard features of PCI Express". GPUDirect RDMA functionality is exposed through the `nv-p2p.h` header distributed in the NVIDIA Driver package, and the modified `e1000e` kernel module must be linked against `nvidia.ko` during compilation.

We allocate some GPU global memory using `cudaMalloc`, called *Packet Memory* (abbr. *PMEM*). This is the chunk of memory in GPU VRAM whose DMA addresses will be configured in the NIC. `nvidia_p2p_get_pages` from `nv-p2p.h` is used to pin the GPU virtual memory corresponding to PMEM, make it accessible to third-party devices through PCI BARs, and to get the GPU page table from which we get the physical addresses of all pages in PMEM.

Next, the `struct pci_dev` corresponding to our NIC is passed to `nvidia_p2p_dma_map_pages` along with our physical addresses to get the DMA addresses we will program into the NIC. This means that the NIC and GPU sitting behind

¹The name is inspired by NVIDIA's GPUDirect libraries.

different IOMMUs is not a problem (which is exactly the case with our setup).

We have arrived at a point where the NIC can be programmed with PMEM's DMA addresses, while at the same time any kernel running on the GPU can also read from the virtual addresses corresponding to PMEM.

B. `e1000_alloc_rx_buffers`

We now have DMA addresses for every byte in PMEM; PMEM is divided into equal sized chunks each of 2048 bytes, with each packet being written to one chunk. We are not using Jumbo Frames hence the frame size of any one packet on the wire is limited to around 1518 bytes; 2048 is thus used as the chunk size. PMEM currently is allocated with a fixed size of 8MB, meaning at any one point it can hold upto 4096 packets without overflowing.

In `e1000e` code, `e1000_alloc_rx_buffers` is used to allocate receive buffers and configure the receive descriptors on the NIC hardware; this includes setting the DMA addresses. The DMA address for each receive descriptor is set individually, and this logic has been modified to instead use DMA addresses corresponding to PMEM chunks. With this change, our packet will now get DMA'd to PMEM!

The PMEM chunks are treated as a ring, simply assigning the chunk corresponding to an index to the receive descriptor currently being programmed inside `e1000_alloc_rx_buffers`, with this index being incremented at the end. This means that if the NIC's receive descriptors are used faster than the PMEMs are processed in the GPU, this PMEM-ring will eventually overflow. We add in no safeguards here except a debug message to indicate if this is happening.

As we will see below, `e1000e` code processes the NIC's receive descriptors in the NAPI poll callback to know when a packet DMA has been completed and is ready for processing. At this point we need a way to get some other info, such as the GPU VA to which this packet has been DMA'd to. Essentially, a way to map the receive descriptor being processing to some other info. Luckily for us `e1000e` code already does this, and one of the pieces of information it maintains is a pointer to a `struct sk_buff* skb` for which it has already allocated memory and it plans on handing over to NAPI receive. Since all packets are being sent to the GPU we have no use for the `skb` buffer here, and as an interesting hack we have overloaded the `skb` pointer to instead point to our own struct with the bookkeeping info.

Finally, the driver continues to function normally until GPUDirect TCP is explicitly enabled; at this point, the driver is hard-reset from within the code to forcefully re-allocate all receive buffers.

C. *Aside: `pkt_desc_t`*

We will soon be communicating packet descriptors everywhere. A packet descriptor is essentially just a combination of an address and a length. The common structure `pkt_desc_t` is used to represent packet descriptors everywhere in our code.

It also contains a magic value as a sanity check, which turns out to be surprisingly useful later.

```
__attribute__((packed)) pkt_desc {
    union {
        void* gpu_va;
        uint64_t offset;
    };
    size_t length;

    uint8_t nic_info[NIC_INFO_SZ];
    uint32_t checksum_verified;
    uint32_t magic;
} pkt_desc_t;
```

D. `e1000_clean_rx_irq`

NAPI invokes `e1000e_poll` which in turn calls `e1000_clean_rx_irq`. Here, the NIC's receive descriptors are looped over, and the descriptors whose Descriptor Done (DD) bit is set are processed. At this point a packet DMA to the GPU has been completed; from the overloaded `skb`, bookkeeping info such as the GPU VA is retrieved and a `pkt_desc_t` is formed. This packet descriptor represents a packet in PMEM which is ready to be processed. What remains now is a method to send this descriptor to the GPU kernel to notify it about this packet.

An interesting point to note is that there is no guarantee in the ordering of PCI transactions issued to different devices. This means that the DD bit could get set by the NIC before the DMA write to GPU memory has completed. This was not an issue when the DMA write also belonged to system memory since PCI guarantees strict ordering for transactions to the same device. We have no numbers to estimate how often this race actually leads to corrupted reads by the GPU kernel.

E. *Sending packet descriptors to the GPU*

All that remains is a notification mechanism to send `pkt_desc_ts` from `e1000_clean_rx_irq` to the GPU kernel. For this, we use an SPSC Queue implemented as a lock-free ring buffer where the producer advances the head and the consumer advances the tail. This specific ring (queue) is called the *RX ring*.

The RX ring is located in another chunk of memory that is allocated in GPU global memory, named *Control Memory* (abbr. *CMEM*). CMEM is allocated in a very similar manner to PMEM, with one difference; we use `ioremap` to make CMEM accessible from the kernel module. Code found inside `init_rx_ring` uses `ioremap` to make the physical addresses we get from GPUDirect RDMA accessible via some data structures, and it also initializes the ring metadata by performing MMIO-writes.

Now, we can push our `pkt_desc_t` to the RX ring from within `e1000_clean_rx_irq`; this will be consumed by the GPU kernel, completing our notification mechanism.

Each `pkt_desc_t` is only 56 bytes in length, and performing MMIO-writes to push the descriptor onto the ring is not an issue when dealing with 1Gbps of throughput. However, our

initial design had packets descriptors of size 128 bytes, and we found a massive gain in performance (from about 600Mbps to line rate) after reducing the packet descriptor size. Hence, these MMIO writes will likely be a bottleneck when pushing for throughputs beyond 1Gbps.

F. Aside: Implementation

`spsc_ring_t` can be found in `gdr_common.h` and is the generic implementation of the SPSC queue in the form of a lock-free ring buffer. The GPU VAs corresponding to CMEM and PMEM are passed to `e1000e` code (`gdr_mem_setup_t`) through a character device after allocating them in the userspace program. `chdev_setup_write()` then calls `enable_gdr()` which is the main startup routine in driver code, which in the end resets the NIC to transition from the normal driver to GPUDirect TCP.

G. Consistency and Data Races

We are now accessing GPU global memory corresponding to CMEM, PMEM through the PCI BARs independent of any CUDA API. To ensure consistency with any CPU-initiated CUDA API operation on the same memory region we set the `CU_POINTER_ATTRIBUTE_SYNC_MEMOPS` attribute as per GPUDirect RDMA documentation².

What is interesting however is that the same documentation states that only "CPU initiated CUDA APIs provide ordering of GPUDirect memory operations as observed by the GPU". The documentation goes on to state:

An example situation ... is when a network RDMA write operation is completed by the third party network device and the data is written to the GPU BAR mapping. ... a concurrently running GPU kernel to that network write might observe stale data, the data partially written, or the data written out-of-order.

A persistent TCP/IP kernel processing our packets is precisely the model we are going for, which "would be considered a data race". Fortunately, we found out that this data is practically never corrupted. As somewhat of a safeguard against this, the GPU kernel when reading a packet descriptor from the RX ring spins till the magic value is valid. It was observed that this magic value is almost never invalid. However, we will see later that the magic value of descriptors in the TX ring, also exposed to the CPU through PCI BARs is found invalid far more often.

III. PACKET REDIRECTION FROM GPU TO CPU

All packets coming into the NIC are now being sent to the GPU. This includes packets which actually belong to applications running on the CPU. For GPUDirect TCP to function transparently we want to redirect these packets back to the CPU and re-insert them into the kernel networking stack, with this path still being fast enough to hit line rate with a minimal increase in latency.

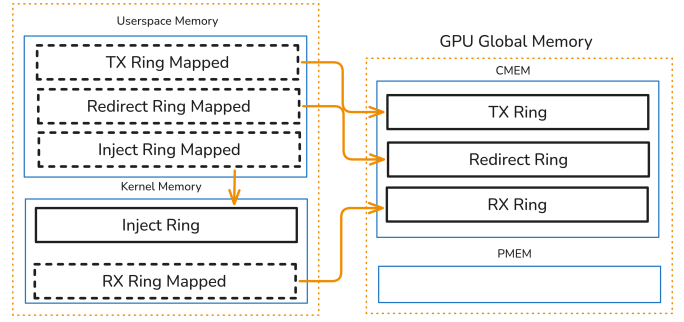


Fig. 1. Summary of various queues and where they are mapped

The biggest bottleneck in transferring packets back from the GPU to CPU is the copying of packets from GPU global memory to system RAM. The first approach we tried was to MMIO map PMEM and copy the packet payloads directly to some kernel memory, from where we construct our sk buffers and hand them off to NAPI. Poor performance was observed (about 40 Mbps) with the bottleneck being the MMIO-copy of packets.

The obvious solution here is to use the GPU's DMA-engine to DMA these packets into system RAM; however, we are forced to use NVIDIA's proprietary driver with our test setup. Thus the GPU's DMA engine cannot be configured directly, and we have no option but to rely on CUDA APIs.

CUDA APIs such as `cudaMemcpyAsync` when used correctly should automatically use the DMA Engine when appropriate. The issue however is that most of the logic so far has been in the `e1000e` driver which belongs to kernel space, and these CUDA APIs cannot be accessed from here. The solution is to use a userspace "relay" program which bridges the gap between `e1000e` and the GPU.

A. Userspace Relay

The userspace relay is implemented in the form of a thread (`gdr_thread()` in `gdrtcp.cu`) which is silently launched by `gdrtcp::init()`. This thread polls the *Redirect Ring* located in CMEM waiting for the GPU kernel to place packet descriptors onto this ring.

To process a packet descriptor, the thread launches a `cudaMemcpyAsync` to copy the packet from PMEM to a chunk of kernel memory called *IMEM* (explained below). Events are inserted into the streams after a certain number of `cudaMemcpyAsync`s to keep track of the completion of these copies.

A successful query on an event indicates that copies scheduled previous to the event on that stream have been completed. The userspace thread now inserts `pkt_desc_ts` into the *Inject Ring* to notify the kernel of these packets, completing the bridge.

Copying the packets is the bottleneck in this path of re-inserting packets back into the CPU. During one of our initial implementations when we were at 200 Mbps, switching from a synchronous `cudaMemcpy` to launching multiple

²<https://docs.nvidia.com/cuda/gpudirect-rdma/index.html>

`cudaMemcpyAsync`s at once along with batched events for multiple copies led to a performance improvement to about 500 Mbps.

B. Redirect Ring

The *Redirect ring* is an SPSC queue also allocated in CMEM like the RX ring. The GPU kernel being the producer pushes packet descriptors for packets not belonging to the GPU workload onto this ring, which are consumed by the userspace relay thread.

To allow the userspace thread to directly access the Redirect ring belonging to CMEM, we map the relevant PFNs obtained from the GPU page table to userspace using `remap_pfn_range`. This is done inside `chdev_redirect_mmap`, which is the `mmap` implementation for the redirect character device. Appropriate flags are set here, such as marking the pages as non-cached.

C. Inject Memory and the Inject Ring

Inject Memory (abbr. *IMEM*) is allocated by `e1000e` using `vmalloc`. IMEM contains space for the Inject ring, its descriptors, and also is used as the destination for packets copied with `cudaMemcpyAsync`.

The userspace relay thread manipulates the Inject ring as well as initiates the `cudaMemcpyAsync`, hence it should be able to access IMEM. For this, we implement an `mmap` on the inject character device, which installs a custom fault handler `chdev_inject_fault`. On a fault, this sets the `struct page` to the same page backing IMEM, essentially mapping our kernel-space IMEM to userspace.

To allow for `cudaMemcpyAsync` to actually DMA packets directly to IMEM without staging copies in some internal buffer, we need to pin IMEM and let CUDA know that this memory is pinned and that staging copies is unnecessary. This is done with the help of `cudaHostRegister`.

Finally, packet descriptors are written to the Inject ring after packet copies have finished, and `e1000e` code can consume these descriptors.

D. `e1000e`: Consuming from the Inject Ring

Where in `e1000e` code should we consume the descriptors which have been pushed to the Inject ring? The simplest answer is within `e1000_clean_rx_irq(work_to_do, ...)`, which is what we have done. We reinterpret the NAPI budget as follows: in each invocation we can process upto `work_to_do` packets from the NIC's receive descriptor ring, as well as the same amount from the Inject ring. If the budget is exhausted with the Inject ring not empty, we keep NAPI in polling mode with interrupts disabled, even if the NIC's actual receive ring is empty.

The actual insertion logic is found in `inject_packets()`. This reads a descriptor from the Inject ring, constructs an sk buffer, copies the packet from IMEM into memory allocated for the sk buffer, and then hands it off to `e1000_receive_skb` which in turn invokes `napi_gro_receive` completing packet re-insertion.

2 extra copies of the packet have been added compared to the usual driver path where the packet is directly DMA'd to memory allocated for the skb; the first copy is from PMEM to IMEM, the second from IMEM to sk buffer memory.

E. When is the NAPI polling callback invoked?

The NAPI polling callback is `e1000e_poll` which calls `e1000_clean_rx_irq`. The invocation of this is hence dependent on packets being received from the actual network, and not that of packets being available in the Inject ring. This implies variance as to when packets from the Inject ring will actually be re-inserted into the kernel networking stack, instead of some interrupt-driven mechanism.

One would not be wrong to think this unpredictability would lead to poor performance, and that it makes little sense to insert packets from the Inject ring based off when other packets are received from the external network. In our testing with an initial prototype where throughput was low due to other reasons, switching from the standard interrupt + polling hybrid NAPI to a continuously polling NAPI (with `ksoftirq` using up 100% of one CPU core) led to an improvement in throughput. It was observed that this was due to a reduction in the jitter of TCP RTT - which is to be expected, since now there was no unpredictability with when the Inject ring was checked.

However, with other improvements and having hit line rate, making NAPI poll continuously was found to be unnecessary to hit line rate. This is likely due to `e1000e_clean_rx_irq` being called often enough due to the higher throughput that this variance in when a packet from the Inject ring will be processed is sufficiently low.

Thus, even though this method of polling the Inject ring makes little logical sense, it works practically with `ksoftirq` cpu usage close to 0%.

IV. TRANSMITTING PACKETS FROM THE GPU

The TCP/IP stack on the GPU may want to send out its own packets onto the wire. For this:

- 1) The *TX ring* is used to send packet descriptors from the GPU kernel to the userspace thread.
- 2) Raw sockets are used to send the packets out onto the wire.

The GPU kernel must construct valid IPv4 packets with headers and valid checksums, whose descriptors it places onto the TX ring.

A. TX ring

Similar to the Redirect ring, the TX ring is allocated in CMEM and exposed to userspace using `remap_pfn_pages` on the GPU PFNs, this time implemented by `chdev_tx_mmap()`.

An interesting observation is that while reading packet descriptors from the TX ring in our userspace thread, the magic value is invalid often enough to be noticeable. Again, we simply spin till the magic value becomes valid. Despite

this, occasionally the checksums included in the copied packet are found to be invalid, in which case the packet is dropped.

The head/tail values, descriptors, and the actual packets all lie on different pages, often separated by large gaps. Although in the GPU kernel we only update the head after writing other information, it is possible that the transactions reading this data are reordered. Adding memory barriers, flushing GPU caches and other attempts to fix this issue did not work. Invalid magic was observed only if the descriptor we were accessing had its VA aligned to 16KB. We are unsure as to what exactly is causing these issues, but luckily the frequency is low enough to impact actual performance. In fact, spinning for even just a single iteration almost always fixes the invalid magic read.

B. Raw Sockets

Raw sockets are used to transmit the IPv4 packets after copying them to a userspace buffer as this is the simplest method. An alternative approach would be for the GPU stack to produce complete Ethernet packets, which would be copied directly to kernel memory and transmitted directly from within `e1000e` code, reducing the various overheads associated with using raw sockets. But, we have optimized our design for receive throughput onto the GPU, and hence for such purposes raw sockets seem to be fast enough to send out ACKs while hitting line rate.

C. Polling in the Userspace Relay Thread

The relay thread consumes packet descriptors from two rings: the Redirect ring and the TX ring. These are ring buffers, and to check for available data the head/tail values of each ring must be checked. The first approach taken was to poll these values continuously, but that for obvious reasons pushes the CPU usage of the thread to 100%.

The natural idea is to let the thread go to sleep when there is nothing for it to do, that is the Redirect and TX rings are empty. The thread calls into a syscall which puts it on a wait-queue, and every invocation of the NAPI callback, specifically `e1000_clean_rx_irq`, wakes up any tasks on the wait queue.

Line rate was observed with this method when majority of the packets coming into the NIC belonged to the GPU, and the CPU usage of the relay thread also dropped considerably; this scenario can be seen specifically in Benchmark 1 below. The other extreme benchmark is that when all packets coming into the NIC belong to the CPU, i.e Benchmark 2 below.

What was observed in this sort of a scenario was that despite the thread going to sleep, CPU usage was maintained at 100%, simply because there are far more packets and a lot more data for the relay thread to transfer. Even worse, the thread going to sleep increased various overheads such as the scheduler overhead, leading to a worse throughput of 700 Mbps compared to line rate being achieved before. Solutions such as extending the amount of time before the thread puts itself to sleep were tried but did not help.

In essence, we are left with the following problem:

- 1) The thread must put itself to sleep frequently if there is less work.
- 2) The thread must not put itself to sleep if there is a lot of work to avoid unnecessary overheads with the scheduler and wait queues.

Developing a proper heuristic for this would involve taking "true" CPU utilization measurements, i.e not including cycles wasted on polling, with differing amounts of work being done by the userspace relay thread. Then, the heuristic would be tuned to put the process to sleep just as frequently as required to reduce CPU usage to this amount where no cycles are wasted on polling. This unfortunately requires taking a lot of measurements and due to a lack of time we have implemented a simple binary heuristic.

Our two extreme scenarios, or benchmarks, involve either a lot of data coming in from the Redirect ring or from the TX ring. The number of packets on the TX ring is far fewer due to the GPU TCP/IP logic, as well as most packets simply being empty ACKs. Hence, this scenario is suitable for the "sleep" mode, whereas when majority of the data comes in through the Redirect ring, we want to poll.

Based off the number of packets coming into the Redirect and TX rings the binary heuristic switches between sleeping and polling mode. This has shown promising results for our two benchmarks, and is basically a proof of concept for a better heuristic.

V. GPU TCP/IP STACK

The GPU TCP/IP stack is implemented almost completely in device-side code, allowing for complete offloading to the GPU. `gdrtcp::init()` launches the `gdrtcp_main` GPU kernel which uses 1 thread block and 64 threads (i.e two warps). The user then launches the workload kernels which can call into the socket-like device-side APIs exposed by GPUDirect TCP, which internally communicate with this persistent `gdrtcp_main` kernel using shared MPMC queues in global memory.

The purpose of the TCP/IP stack is two-fold:

- 1) To reconstruct from received TCP/IP/Ethernet packets a byte stream of data which the user will read from.
- 2) To convert the byte stream of data the user wants to send into TCP/IP packets and transmit them.

At this point, we can see the dividing line between the user and the TCP/IP stack consists of the send and receive buffers. The TCP/IP stack processes packets and writes to the receive buffer, which is then read from by the user (using some library-provided functions). The user writes to the send buffer (again using some library functions), which then the TCP/IP stack converts to packets and sends out onto the wire.

Thus this section is approached in these two parts, with both of them being mostly independent of the other.

A. Some Key Data Structures

Most of these structures are found in `gdrtcp.h`. `connection_t` is used to represent a single connection;

some example fields are shown below. This is information which the user does not care about.

```
struct connection_t {
    volatile bool syn_sent;
    volatile bool established;
    ...
    __be32 peer_ip;
    __be16 peer_port;
    ...
    volatile uint32_t last_ack_sent;
    volatile uint32_t last_ack_recv;
    ...
    volatile uint32_t rwnd;
    volatile bool send_doorbell;
    socket_t socket;
    ...
};
```

socket_t is the bridge between the user and the library; it consists of the send and receive buffers.

```
struct socket_t {
    volatile rx_ring_t rx_ring;
    volatile tx_ring_t tx_ring;
    connection_t* conn;
    ...
};
```

A TCP "environment" is allocated in global memory which maintains global state, such as the connection table containing all connections from the GPU.

```
struct tcp_env_t {
    op_queue_t submission_queue;
    op_queue_t completion_queue;
    connection_t conn_table[...];
    int exiting;
    ...
};
```

B. Overall Structure

The processing of packets being received is completely handled by `recv_warp`. As the name suggests, the processing of packets is parallelized over 32 threads. Although one would expect the GPU to be worse at TCP processing than a CPU, we will see how having many cores helps optimize the bottleneck. Broadly, `recv_warp` does the following:

- 1) Redirect packets not belonging to the GPU back to the CPU.
- 2) Process SYN/ACKs and FIN/ACKs.
- 3) Process packets belonging to established connections, and copies their payloads to the receive buffers.
- 4) Buffers Out-of-Order (OOO) packets.

The design of `recv_warp` is fairly mature and we believe the approaches taken are fairly optimal. This is in contrast to `send_thread`, which implements very simple sending logic.

As suggested by the name, `send_thread` is not parallelized. Additionally, it does not support standard features such as TCP congestion control. Broadly, `send_thread` does the following:

- 1) Sends SYNs and FINs.
- 2) Sends ACKs when kicked by `recv_warp`.
- 3) Sends data the user wants to transmit.
- 4) Implements simple retransmission logic, where unacknowledged packets are retransmitted at a constant interval.

Both `recv_warp` and `send_thread` are divided into "Parts". For `recv_warp`, `__syncwarp()` is invoked after each part to symbolize that each part is intended to be executed by all threads of the warp in lockstep, although this would anyway happen without the explicit call.

C. Recv Warp: Part 1, Waiting

Each thread of the `recv_warp` processes one received packet. In Part 1, we simply wait till there are 32 packets descriptors available in the RX ring which is written to by e1000e. Each thread then processes a packet according to its lane.

This method of waiting leads to poor performance when the rate of packets being received is low, as certain packets may have to wait for long before being processed. This was particularly observed during TCP's slow start; to solve this, a hybrid strategy is used instead. If at least one packet is available, then we wait no more than `MAX_WAIT_ITERATIONS` before moving on to Part 2. The current value is set to 20 and it works well enough.

D. Recv Warp: Part 2, Classification and Header Parsing

First, each thread checks if its packet is an Ethernet, IPv4, TCP packet. If it is, then it loops over the connection table to check if the packet belongs to a GPU connection. If a match is found the below processing is done; otherwise, the thread does nothing till Part 3.

If the packet is a SYN/ACK or a FIN and the connection is in the `SYN_SENT` or `FIN_WAIT_1` state, the packet is completely processed here. Fields corresponding to the connection are updated and the state of the connection is changed, with `send_thread` being kicked to send an ACK.

Else, we now know that this packet contains some data which must be copied into the receive buffer which the user will access. For this, an offset into the receive buffer is calculated which maps to the sequence number of this packet. Other relevant fields are also extracted from the TCP header.

We now have an offset into the receive buffer for the payload of this packet. We do not care if this packet is OOO, just that the offset where the payload will be copied to should not overwrite data which has not yet been read by the user. A packet being so far OOO is unlikely, but if it happens to be the case, the packet is simply dropped.

E. Recv Warp: Part 3, Copying to the Receive Buffer

For packets belonging to a GPU connection, the payload must be copied to the receive buffer. This is a copy which

cannot be avoided. This is also the *only* copy performed by GPUDirect TCP for packets belonging to a GPU-connection.

This is the bottleneck in `recv_warp`. To perform the copy, each byte of the payload is iterated over and assigned to the correct offset inside the receive buffer. Because every thread copies its payload in parallel, considerable performance gains were observed, and this parallelization was crucial to hitting line rate with the GPU TCP stack.

Threads processing packets not belonging to a GPU connection do nothing in Part 3.

F. Recv Warp: Part 4, Redirect Packets, Update State

This part is completely serialized and executed only by the first thread in the warp due to the difficulty in meaningfully parallelizing this logic. Nevertheless, this is not the bottleneck.

Firstly, packet descriptors for packets not belonging to a GPU connection are pushed to the Redirect ring. Next, we are looking to calculate the new cumulative ACK for each GPU connection which received packets. The last ACK sent is stored in the `connection_t` for the connection. Insertion sort is used to sort the sequence numbers of the 32 packets, and we simply iterate over these sorted `[start, end]` pairs to update the cumulative ACK. Similarly, the highest ACK received for the connection is updated which is needed by `send_thread` to know which data has been acknowledged.

The number of bytes the cumulative ACK to be sent out has advanced by is used to advance the head of the receive buffer, and the receive window advertised by the GPU stack is also reduced by the same amount. This is crucial to apply backpressure onto the TCP connection if the user is not consuming data as fast as it is being received. Finally, the send doorbell of the connection is rung, which kicks `send_thread` causing it to send an ACK. This ACK is sent even if the cumulative ACK has not advanced, which will eventually trigger fast retransmit on the sender's side as desired.

G. Recv Warp: Part 4, Out-of-Order Buffering

(This code also belongs to the Part 4 but we have split the explanation into two parts)

Note that in Part 3 we simply copy the packet payload to the appropriate offset inside the receive buffer, without checking if the packet is in-order. Hence, OOO packets are already buffered implicitly. All we need to do is maintain the sequence number segments corresponding to OOO packets which we have received, and merge these segments when the holes are filled by newly received packets.

The sequence number segments covered by previously-received OOO packets are maintained in certain structures inside `connection_t`. We merge these segments with OOO segments in the current batch of packets and sort them. Finally these OOO segments are iterated over, and if the OOO segments directly succeed the current cumulative ACK, these segments are popped and the cumulative ACK is accordingly advanced.

A TCP implementation should be able to handle a variety of edge cases, with overlapping packets being one. Overlapping packets are unlikely except in the situations of heavy packet loss when there are a lot of retransmission, which is also the scenario where OOO packets become more likely. We however do not handle this edge case of overlapping segments, causing certain OOO segments to never get flushed naturally. Thus OOO segments more than a few batches old are simply deleted.

H. Send Thread: Part 1, Waiting

The send thread spins until one of the following events happen for any connection:

- 1) The send doorbell is rung.
- 2) An operation is submitted on the submission queue.
- 3) RTO fires.

The submission queue, covered later, is the queue on which the user can submit commands to either connect a given socket to a remote peer or to close an established connection.

I. Send Thread: Part 2, Processing the Event

If the event corresponds to a user wanting to establish a connection, a SYN is formed with a few additional options. Similarly, a FIN is formed if the connection must be closed.

If the event is the send doorbell being rung, we form a packet with the latest ACK to be sent and fill out the payload if any data is available in the send buffer. Our IPv4 packets have the Don't Fragment bit set, and as the GPU stack does not perform path MTU discovery, we opt to send a max payload of 536 bytes per packet which is guaranteed to be supported by all network links.

If the event is an RTO firing, we set some variables to point to the first packet in the retransmit queue maintained in `connection_t`.

J. Send Thread: Part 3, Send the Packet Out

To send the packet out onto the wire, a packet descriptor is formed and pushed onto the TX ring. The rest of the work will be done by the userspace relay program.

Newly sent packets are also pushed onto the retransmit queues. When retransmitting packets, we send the original packet again, instead of reconstructing a packet with the same payload. This is unusual since by the time of retransmission certain fields in the header such as the `ack_seq` could be stale, but it is simple and works.

K. User APIs

The TCP/IP stack now reconstructs the TCP byte stream in the receive buffer and sends packets corresponding to data placed in the send buffer. What remains is a way for the user to interface with these buffers, and to communicate requests to open and close connections. Recollect that the TCP/IP kernel is a persistent kernel different from kernels the user can launch; as shown below, the user can use device-side wrappers to communicate with the TCP/IP kernel.

First, opening and closing connections: the user can call `socket_connect(..., struct sockaddr_in)` and `socket_close(...)`. These functions send requests to the TCP/IP kernel over an MPMC queue in global memory, called the *Submission Queue*. The TCP/IP kernel can return any data back using another MPMC queue, named the *Completion Queue*.

The critical function performed by the user is to read and write data to and from the socket buffers. The requirements of these interfaces is as follows:

- 1) The user should be able to access and send data without creating unnecessary copies.
- 2) The interface should allow the user to receive/send at a high throughput while keeping it easy to split the data between multiple threads.

With these constraints, the classical *NIX-like socket interface makes little sense; instead, we opt for the below design.

`slice_t recv_slice(..., min_len, max_len)` is called to receive a "slice" of data. Once the user has processed the data in the slice, it must invoke `recv_release(slice)`. A slice is simply a chunk in global memory:

```
typedef struct _slice {
    volatile void* addr;
    int length;
} slice_t;
```

The user is expected to invoke one `recv_slice` per thread block and then share the slice with other threads through shared memory; usage would look like:

```
__shared__ slice_t slice;
if (lane == 0)
    slice = recv_slice(min_len, max_len);
__syncthreads();

do_work_at_my_offset(slice, lane);

__syncthreads();
if (lane == 0)
    recv_release(slice);
```

Internally, `recv_slice` just returns a pointer pointing to an offset in the receive buffer and advances an internal counter for the next `recv_slice`. `recv_release` frees up the space in the receive buffer and increases the connection's receive window, which was reduced when data was written to the receive buffer. With this, we have a form of TCP flow control and backpressure is applied if the application does not process data as fast as it is being sent.

The receive and send buffers are implemented yet again as a ring. If the user requests data which is partially at the end of the ring and partially at the beginning, we cannot return such a slice, as a slice is a contiguous $[addr, addr + len)$. To resolve this, the returned slice is always a power of two which divides the size of the ring. The user can thus only receive slices in

powers of two which is a weird constraint, but the rings are big enough that for any practical purpose this is not an issue.

The sending side interfaces are similarly designed, with `send_reserve` reserving a slice of memory in the send buffer which it can directly write to, and `send_complete` transferring ownership of this slice to the TCP/IP kernel.

In summary, with this design the two required constraints are satisfied. The only copy in the receive path is from the packets to the receive buffer, and the receive buffer can now be accessed directly by the user in a parallel fashion.

VI. IMPLEMENTATION LIMITATIONS

A. Launching Concurrent Kernels

As mentioned above, the TCP/IP Kernel is designed to be launched independently to user kernels. The kernels should be able to run concurrently on different GPU SMs provided they are launched on differing (non-default) streams. This behavior is suggested by various pieces of NVIDIA documentation³ and online examples.

Weirdly, we could not get concurrent kernel launch to work; the TCP/IP Kernel is launched when `gdrtcp::init` is invoked, and is designed to run until `gdrtcp::destroy`. The user is expected to launch their workload kernels in between which would communicate with the TCP/IP Kernel to do their work. It was observed that the user kernel is never scheduled by the GPU while the TCP/IP Kernel is running.

We ran minimal examples to confirm this behavior, on the Quadro P620 as well as an RTX 3050 in a laptop. This issue destroys the clean interface of GPUDirect TCP.

Nevertheless, to continue testing, we have hardcoded the launch of the user's kernel inside a different thread block. That is, a single kernel is launching with two thread blocks - one running the user's workload, the other running the TCP/IP logic. Since threads in different thread blocks are forced to communicate with each other through global memory and can also be scheduled on different SMs, from a testing perspective, this is equivalent to launching two separate kernels. Well, except being a not-so-pretty sight.

B. Supporting Multiple GPU Connections

Most parts of the implemented logic contains support for multiple GPU connections, as the design explains above. However the MPMC Submission and Completion queues are SPSC in reality. This means that multiple connections are actually not supported although the design and code in most places has the required logic, and we have not tested with multiple GPU connections.

VII. BENCHMARKING

All benchmarks use a common benchmark TCP server, implemented in `bench_server.c`, to which a benchmark client connects. The logical flow of all benchmarks is as follows:

³An example: <https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/asynchronous-execution.html#legacy-default-stream>

- 1) The client connects to the server.
- 2) The client sends the amount of data it wants to receive for this benchmark, called `test_size`.
- 3) The server proceeds to pump in randomized data into the connection using standard *NIX socket APIs, until `test_size` bytes have been sent.
- 4) The client sends a checksum calculated over all the received data which is used to verify correct reception of the randomized data.
- 5) Client closes the connection.

Benchmarking has two purposes: to check correctness of the TCP implementation and to measure performance.

A. Correctness

The checksum is used to verify if the data received by the client was correct and in the right order. This is specifically important to verify if the byte stream reconstructed by the GPU TCP/IP stack is valid. The checksum used is simply a sum over the product of each byte multiplied with it's position in the byte stream.

Benchmarks with a `test_size` of 32 GB were used to test for correctness. With randomized data being sent, valid checksums on tests of this size is sufficient to verify correctness.

B. Performance

Our test setup involved sending data between two gigabit NICs on the same computer, artificially separated by placing the interfaces into two separate network namespaces to force the packets to go to wire. It is expected for such a setup to have low RTT, low packet loss and a low number of OOO packets. A proper benchmark would also usually involve core isolation, pinning the programs, etc. which we have not done.

The server measures and saves various performance indicators. A point to note is that on a gigabit ethernet connection, a full frame is 1538 bytes whereas the TCP MSS is 1460 bytes.

$$\frac{1460}{1538} \times 1000 = 949.2 \text{ Mbps}$$

That is, 949 Mbps is the line rate for TCP throughput in our test setup.

C. Benchmark 0: Without GPUDirect TCP

To get a feel of what the numbers should look like, we run a TCP client (`bench_native.c`) running on the CPU which uses the *NIX socket APIs; GPUDirect TCP is not enabled.

Ballpark numbers are observed in all benchmarks as exact numbers vary from run-to-run.

Throughput	Line Rate
RTT Variance	100-200 us
RTT	2100 us
Server's Congestion Window	460 KB
CPU Usage: Total	25%
CPU Usage: ksoftirq	0%

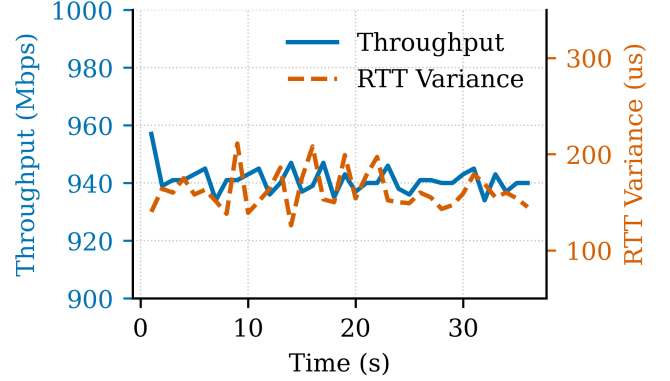


Fig. 2. Benchmark 0: Throughput and RTT variance

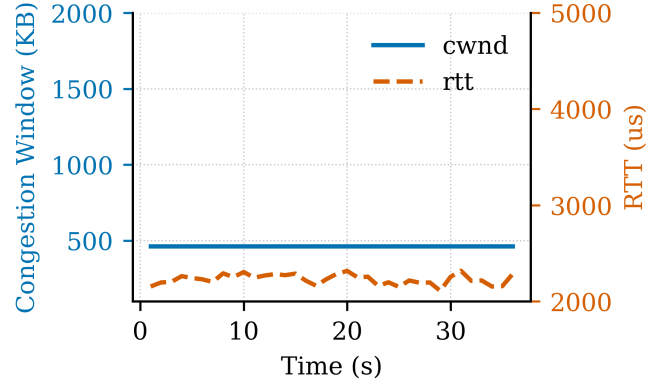


Fig. 3. Benchmark 0: Congestion Window and RTT

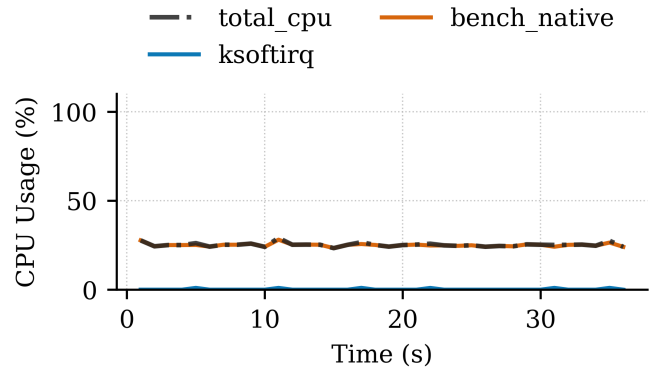


Fig. 4. Benchmark 0: CPU Usage

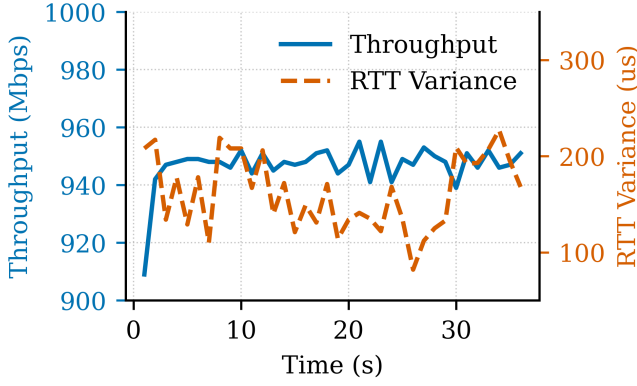


Fig. 5. Benchmark 1: Throughput and RTT Variance

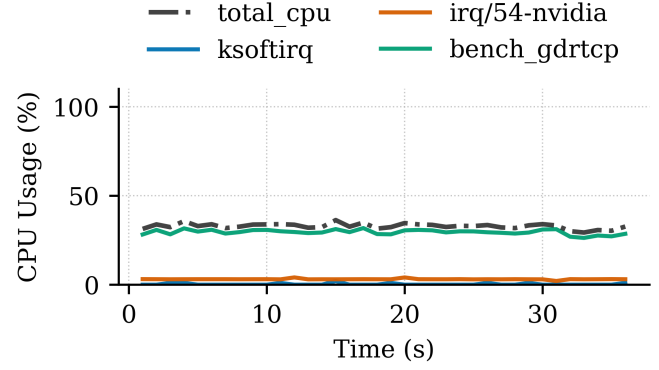


Fig. 7. Benchmark 1: CPU Usage

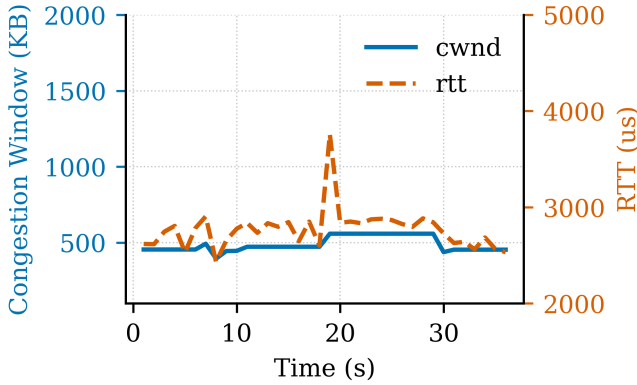


Fig. 6. Benchmark 1: Congestion Window and RTT

An interesting point to note is that the TCP client receiving the packets and calculating the simple checksum uses up 25% of one CPU core at line rate.

VIII. BENCHMARK 1: GPU TCP/IP STACK

A. Setup

`bench_gdrtcp` implements a TCP client running on the GPU TCP/IP stack. This connects to the server and requests 4 GB of data. `benchmark_client()` in `gdrtcp.cu` implements the actual logic; it calculates the checksum in parallel over 64 threads in one thread block to be able to keep up with 1Gbps of data.

B. Results

Throughput	Line Rate
RTT Variance	100-200 us
RTT	2800 us
Server's Congestion Window	500 KB
CPU Usage: Total	33%
CPU Usage: ksoftirq	0%
CPU Usage: irq/54-nvidia	3%
CPU Usage: bench_gdrtcp	30%
Invalid TX Checksums:	≈ 10

C. Observations

RTT variance remains the same as Benchmark 0, with RTT going up a bit. RTT variance matters more than one probably expects - to get to line rate, multiple optimizations were made to reduce jitter in ACK timings. With high variance in RTT, throughput suffers as CWND takes time to "catch-up" to the new RTT.

Overall CPU usage sits at around 33%, with 3% coming from the GPU interrupt handler and the CPU usage from ksoftirq still almost at 0%. This is both good and bad.

The bad part is that despite aiming to offload TCP/IP processing completely to the GPU, CPU usage sits at 33%. The good part is where this CPU usage comes from.

We cannot modify our GPU driver's code, forcing the workaround of the userspace relay which is eating up 30% of one core. A part of this is also likely coming from our simplistic usage of raw sockets. But these observations combined with Benchmark 2 show that complete offloading to the GPU while hitting line rate is possible, despite our implementation not achieving 0% CPU usage. See Benchmark 2 below for the complete reasoning.

Finally, in section IV.A it was noted that despite valid magic in the descriptor, some packet checksums are found to be invalid after being copied to RAM. This count is printed to the terminal by the client as a part of "CPU Stats" and is often in the high single-digits for tests of a few GB.

IX. BENCHMARK 2: GPU TO CPU PACKET REDIRECTION

A. Setup

GPUDirect TCP is enabled but there are no active connections belonging to the GPU. `bench_redirect.c` connects to the server using standard *NIX sockets and requests 4 GB of data. This benchmark tests the performance of applications running on the CPU when GPUDirect TCP is enabled, essentially stress-testing the redirect and inject mechanisms.

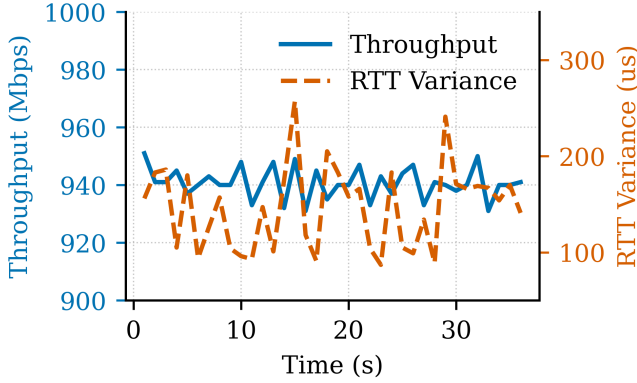


Fig. 8. Benchmark 2: Throughput, RTT Variance

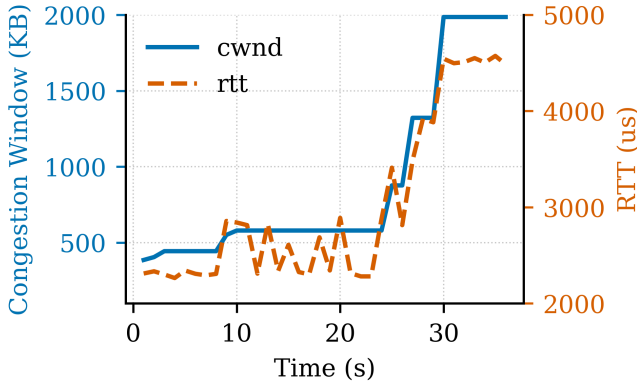


Fig. 9. Benchmark 2: Congestion Window, RTT

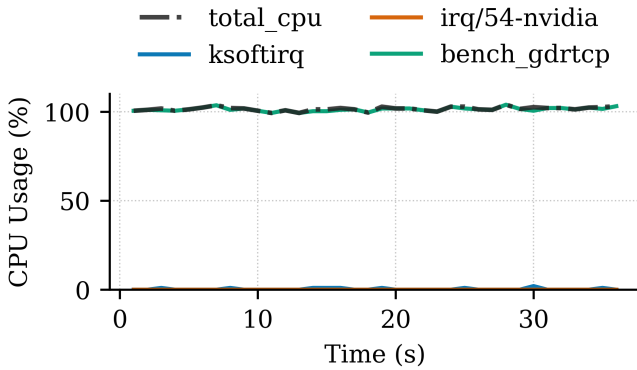


Fig. 10. Benchmark 2: CPU Usage

B. Results

Throughput	Line Rate
RTT Variance	100-200 us
RTT	2000-5000 us
Server's Congestion Window	500-2000 KB
CPU Usage: Total	102%
CPU Usage: ksoftirq	0%
CPU Usage: irq/54-nvidia	0%
CPU Usage: bench_gdrtcp	102%

C. Observations

It was observed across multiple experiments that RTT goes upto 6-7 ms, with CWND scaling accordingly. This is to be expected since now we have introduced 2 extra copies before each packet is even inserted back into the linux networking stack. Keeping buffers large enough and RTT variance low however allows for throughput to hit line rate.

CPU usage crosses 100%; CUDA spawns its own threads which consume a few percent, and our userspace relay thread has to transfer huge amounts of data leading to high CPU consumption. This is in contrast to Benchmark 1, where the userspace relay copied fewer packets due to the GPU TCP/IP stack generating 1 ACK for every batch of packets it processes. Additionally, almost every packet in Benchmark 1 was just an empty ACK with no payload.

D. Eliminating the Userspace Relay for 0% CPU Usage

Benchmarks 1 and 2 combined show us that complete offload to the GPU is possible provided that one can modify the GPU driver to perform GPU-initiated DMA's to kernel memory.

First, let's talk about Benchmark 2. `ksoftirq` usage is 0% in this benchmark; if the GPU can directly DMA the packets into IMEM and push packet descriptors to the Inject ring, the userspace relay is completely eliminated. `e1000e` code logic is completely unchanged, and there is no reason to doubt that 0% CPU Usage would be achieved with these modifications.

Coming to Benchmark 1: firstly, the number of packets and the actual amount of data being sent back to the CPU in Benchmark 1 is significantly lower than Benchmark 2. To eliminate the userspace relay, the GPU would have to DMA packets and place packet descriptors on a new ring which is directly accessible by `e1000e`. `e1000e` transmit code would have to be modified to send out these packets, replacing the raw sockets being used right now.

As `ksoftirq` CPU Usage in Benchmark 2 is 0% even with a significantly higher number of packets and data being processed by the driver code, it makes logical sense that this newly modified `e1000e` transmit code would also be able to comfortably keep up with whatever rate of packets the GPU wants to transmit.

Hence, from Benchmarks 1 and 2 we believe that complete offloading to the GPU is easily doable with these changes.

X. CONCLUSIONS AND WORK LEFT

A. Uses

Benchmarks 1 and 2 have shown that full TCP offload to the GPU is possible with 0% CPU usage.

This, in fact, is slightly surprising to us. We did not expect a TCP stack on the GPU to be able to hit line rate. But it seems like 1Gbps of throughput is less than what it sounds like. Pushing to higher throughputs with more sophisticated hardware is required to make any meaningful comparisons.

At the same time if one is willing to invest in expensive high throughput NICs, NVIDIA already sells ConnectX NICs with inbuilt GPUDirect RDMA support. We are unsure of the specifics, but it seems like CPU offloading is a solved problem when it comes to specialized NICs and GPUs used in data centers.

So what is left is commodity hardware, and it seems unrealistic that one is looking to run such workloads on commodity hardware that even a single CPU core cannot be spared. The only exception to this we can think of is a commodity GPU-cluster, where GPUs significantly outnumber available compute. GPUDirect TCP in such a scenario could allow for direct GPU-GPU communication on top of GPU's directly being able to communicate with external peers.

B. Work Left

The sending side logic of the GPU TCP/IP stack is extremely simple, and a parallelized implementation can be done here similar to the receive logic. However, the sending side logic in TCP is far more complex, and is unnecessary if the GPU is mostly just receiving data.

The concurrent kernel launch issue needs be solved to actually make GPUDirect TCP a proper library. Parts of the actual implementation also need to be modified to support multiple connections from the GPU.

Trying the changes suggested in IX.D is probably the most interesting next step that can be taken.

XI. APPENDIX

A. Artifact

Please read `README.md` for all information related to running the artifact. If you use machine 249 (which is our machine), please remove the NVIDIA kernel modules we have installed in `/lib/modules/6.1.4/extra/`; care must be taken to uninstall other NVIDIA stuff which is installed globally by the installer (we ourselves are unsure of these uninstall steps).

The simpler option is to just login to our user account (divyansh) and run tests from here.

B. Attribution

AI Tools were used to generate plotting code and scripts. All other code has been written by us except large chunks of code in `helpers.cuh`; mostly byte order conversions, TCP/IP checksum calculation, and converting packed TCP/IP headers to non-packed versions (some bugs were observed when using bitfields in the standard packed headers in device code).